Production-Level Web Application Development Guidelines

1. Purpose

Before starting development of the client portal application, the entire project must follow a standardized, production-ready development structure.

The objective is to ensure:

- Clean and maintainable code
- Consistent UI/UX
- Reusable components
- Minimal code duplication
- High application performance
- Optimized API usage
- Reduced network and database overhead
- Reduced API egress
- Strong security
- Scalable architecture
- Easy debugging and maintenance
- Consistent development practices across all modules

These guidelines should be treated as **global project standards** and followed throughout development.

---

2. Project Architecture & Planning

Before writing application code:

- Define the overall application architecture.
- Identify frontend, backend, database, storage, authentication and third-party integrations.
- Define the responsibilities of each layer.
- Avoid mixing business logic, UI logic and API logic unnecessarily.
- Establish common development patterns before creating individual modules.
- Define naming conventions globally.

- Define reusable components and utilities before creating duplicate implementations.
- Document important architectural decisions.
- New modules must follow the existing architecture instead of introducing a new pattern without justification.

Architecture Principle

Follow:

UI → Components → Hooks → Services/API → Backend → Database

Avoid:

UI → Direct database/API logic → duplicated business logic

---

3. Proper Folder & File Structure

The project must follow a predictable and scalable folder structure.

Example:

```
src/
├── assets/
├── components/
│   ├── common/
│   ├── forms/
│   ├── tables/
│   ├── modals/
│   └── navigation/
├── pages/
├── layouts/
├── hooks/
├── services/
├── utils/
├── constants/
├── context/
├── store/
├── styles/
├── validations/
├── types/
└── config/
```

Backend:

```
backend/
├── routes/
├── controllers/
├── services/
├── repositories/
├── models/
├── schemas/
├── middleware/
├── utils/
├── config/
└── tests/
```

Rules

- Do not place everything inside one file.
- Do not create unnecessary folders.
- Files must have a single clear responsibility.
- Avoid extremely large components/files.
- Related functionality should remain grouped together.
- Common functionality must be moved to reusable modules.
- Avoid circular dependencies.
- Do not create duplicate utility functions.

---

4. Reusable Components & DRY Principle

Follow the **DRY — Don't Repeat Yourself** principle throughout the application.

Reusable components should be created for commonly used UI and functionality.

Examples:

- Search bar
- Filter component
- Buttons
- Dropdowns
- Input fields
- Textareas
- Modals
- Confirmation dialogs

- Tables
- Pagination
- Loading indicators
- Empty states
- Error states
- Navigation
- Sidebar
- Header
- Layout
- Cards
- Form components
- File upload component
- Date picker
- Toast/notification
- Status badges

Important Rule

If the same UI/functionality is required in multiple places, **reuse the existing component instead of creating another version**.

Avoid:

SearchBar.jsx
UserSearchBar.jsx
ClientSearchBar.jsx
ProjectSearchBar.jsx

Prefer:

SearchBar.jsx

with configurable properties.

The same principle applies to:

- Functions
- Hooks
- API services
- Validation
- CSS
- Constants
- Utilities

---

5. Component Design Standards

Every component should have a clear responsibility.

Avoid creating components that perform all of the following:

- UI rendering
- API requests
- data transformation
- validation
- business logic
- state management
- error handling

Instead, separate responsibilities.

Example:

Component
↓
Custom Hook
↓
API Service
↓
Backend API

Component Rules

- Keep components small and readable.
- Use props for configurable behavior.
- Avoid deeply nested components.
- Avoid unnecessary re-renders.
- Do not duplicate components for minor UI differences.
- Use composition where appropriate.
- Avoid hardcoding business values inside UI components.

---

6. Custom Hooks

Custom hooks must be used whenever reusable stateful logic appears in multiple places.

Examples:

useSearch()
useDebounce()
usePagination()
useFetch()
useModal()
useForm()
useFileUpload()
useAuth()
usePermissions()
useApiRequest()

Custom hooks should be responsible for reusable logic, while components should primarily handle presentation.

Avoid duplicating:

useState()
useEffect()
API calls
loading logic
error logic
pagination logic

across multiple pages when the behavior is essentially the same.

---

7. Global Constants & Configuration

Frequently used values must not be hardcoded throughout the application.

Examples:

API URLs
Pagination limits
Debounce delays
File size limits
Allowed file types

Timeout values
Status values
Role names
Permission names
Date formats
Validation limits

Use centralized configuration:

constants/
config/

Example:

PAGINATION_LIMIT = 20
SEARCH_DEBOUNCE = 400
MAX_FILE_SIZE = ...

If a global value changes, it should be possible to update it from one location.

---

8. UI/UX Consistency

The entire application must follow a consistent visual system.

The following must remain consistent globally:

- Typography
- Font sizes
- Font weights
- Colors
- Border radius
- Shadows
- Buttons
- Inputs
- Dropdowns
- Tables
- Cards
- Spacing
- Padding

- Margins
- Icons
- Loading states
- Error states
- Empty states
- Modal behavior

Do not create visually different versions of the same component without a valid UX requirement.

---

9. Global CSS & Design System

Create a centralized design system.

Define global values for:

Colors
Font family
Font sizes
Font weights
Spacing
Border radius
Shadows
Breakpoints
Transitions
Z-index
Container widths

Example:

--spacing-xs
--spacing-sm
--spacing-md
--spacing-lg

--radius-sm
--radius-md
--radius-lg

Avoid random values throughout the application.

Instead of:

margin: 13px;
padding: 17px;

use standardized spacing whenever possible.

---

10. Responsive Design

The application must be responsive from the beginning.

Support appropriate:

- Desktop
- Laptop
- Tablet
- Mobile

Do not develop desktop-only UI and attempt to fix responsiveness at the end.

Important areas:

- Navigation
- Tables
- Forms
- Modals
- Cards
- Filters
- Search
- Pagination
- Sidebars
- Dashboards

---

11. Forms & Input Validation

Every input field must have defined limits.

For each input, determine:

- Minimum length
- Maximum length
- Allowed characters
- Required/optional status
- Data type
- Validation rules
- Error message

Examples:

Name → min/max length
Email → valid email format
Phone → valid phone format
Description → maximum character limit
Textarea → maximum characters
Number → min/max value

Validation must exist at both:

Frontend + Backend

Frontend validation improves UX.

Backend validation provides security and data integrity.

Never rely only on frontend validation.

---

12. File Upload Guidelines

File uploads must have strict controls.

Define:

- Maximum file size
- Minimum file size where required
- Allowed extensions
- Allowed MIME types
- Maximum number of files
- Filename restrictions
- Storage location
- Upload timeout

- Duplicate handling

Example:

PDF → application/pdf
DOCX → application/vnd.openxmlformats...
PNG → image/png

Do not trust only the file extension.

Validate file type on the backend as well.

Large files should not unnecessarily pass through the application server if direct/object storage upload is appropriate.

---

13. Search & Filter Guidelines

Search and filtering must follow a globally consistent pattern.

Debouncing

Do not trigger an API request for every keystroke.

Use a standardized debounce delay.

Recommended starting point:

300–500 ms

A common default:

400 ms

The exact value should be standardized globally and changed only when there is a specific UX/performance reason.

Search Rules

- Do not request data when the query has not meaningfully changed.
- Trim unnecessary whitespace.

- Avoid duplicate requests.
- Cancel obsolete requests where appropriate.
- Reset pagination when search criteria changes.
- Preserve relevant filter state.
- Avoid sending unnecessary parameters.

Example:

User types:

a
ab
abc
abcd

The API should not necessarily receive four requests.

Instead, after the user pauses typing:

abcd → API request

---

14. Filter Guidelines

All filter components must follow consistent behavior.

Define globally:

- Default page size
- Dropdown loading behavior
- Search behavior
- Empty state
- Clear behavior
- Multi-select behavior
- API parameter format
- Sorting behavior

For example, if dropdown APIs use:

20 records

as the standard initial limit, all equivalent dropdowns should follow the same convention unless there is a documented reason otherwise.

Do not randomly use:

10
20
25
50
100

without a reason.

---

15. API Pagination Standards

All APIs returning potentially large datasets must use pagination.

Prefer server-side pagination.

Example:

page
limit
offset

or preferably cursor-based pagination where appropriate for large or frequently changing datasets.

Standard API Limit

Define a global default.

Example:

Default page size: 20
Maximum page size: 100

The exact values should be selected based on application usage and performance testing.

Important

Do not return thousands of records when the UI only displays 20.

Bad:

API → 5,000 records
Frontend → displays 20

Better:

API → 20 records
Frontend → displays 20

---

16. Pagination Consistency

Pagination behavior must be consistent across the application.

Define globally:

- Default page size
- Maximum page size
- Page navigation behavior
- Total count strategy
- Cursor/offset strategy
- Sorting parameters
- Filter parameters

When filters/search change:

Reset → page 1

Avoid inconsistent pagination implementations between modules.

---

17. API Architecture

API requests must be centralized.

Avoid writing raw API requests repeatedly inside components.

Instead:

Component
  ↓
Custom Hook
  ↓
API Service
  ↓
Backend

Example:

services/
  usersService.js
  projectsService.js
  clientsService.js

This provides:

- Consistency
- Easier maintenance
- Centralized authentication
- Centralized error handling
- Easier testing
- Easier API changes

---

18. API Request Cancellation

Use `AbortController` where cancellation is meaningful.

Important use cases include:

Search

Cancel an older search request when a newer search request is initiated.

Navigation

Cancel requests for data that is no longer required after navigating away.

Component unmount

Prevent unnecessary state updates from obsolete requests.

Cancel button

Allow users to cancel long-running operations where supported.

Rapid filter changes

Cancel previous requests when the user changes filters quickly.

Do not add abort logic everywhere blindly.

Use it where it provides actual benefit.

---

19. API Request Deduplication

Prevent duplicate API requests.

For example, avoid:

Same request
Same request
Same request

being sent simultaneously because of multiple renders or effects.

Possible strategies:

- Request caching
- Query libraries
- Request keys
- Debouncing
- Memoization
- State-based request control

---

20. API Error Handling

Every API request should have predictable states:

Loading
Success
Empty
Error
Cancelled
Retry

The frontend must not silently fail.

Display meaningful messages to users while keeping technical details out of the UI.

Example:

Bad:

AxiosError: 500 Internal Server Error

Better:

Unable to load projects. Please try again.

Technical details should be available in logs for developers.

---

21. API Timeout & Retry Strategy

Define appropriate API timeout behavior.

Not every request should retry automatically.

Use retries carefully for:

- Temporary network failures
- Certain 5xx responses
- Transient third-party failures

Do not automatically retry:

- Validation errors
- Authentication errors
- Permission errors
- Invalid requests

Retry policies should prevent request storms.

---

22. API Performance & Egress Optimization

API design must minimize unnecessary data transfer.

Avoid

SELECT *

when only a few fields are required.

Return only required fields.

Example:

Instead of:

```
{
  "id": 1,
  "name": "...",
  "email": "...",
  "address": "...",
  "createdAt": "...",
  "updatedAt": "...",
  "internalData": "...",
  "metadata": "..."
}
```

return only what the UI requires.

{

```
  "id": 1,
  "name": "...",
  "email": "..."
}
```

This reduces:

- Network transfer
- API response size
- Database workload
- Serialization cost
- Frontend processing
- Egress usage

---

23. Reduce Database Egress

Do not repeatedly download the same data unnecessarily.

Use:

- Server-side filtering
- Server-side pagination
- Field selection
- Caching
- Request deduplication
- Aggregation
- Incremental loading
- Appropriate indexes

Avoid fetching complete datasets simply to filter them in the browser.

---

24. Database Query Optimization

Database queries must be optimized before production.

Important areas:

- Proper indexes

- Avoid N+1 queries
- Avoid unnecessary joins
- Select only required columns
- Use pagination
- Use appropriate query conditions
- Analyze slow queries
- Avoid unnecessary repeated queries
- Use transactions where required

Frequently searched/filterable fields should be evaluated for indexing.

---

25. Caching

Use caching where appropriate.

Potential candidates:

- Static configuration
- Frequently accessed reference data
- Dropdown values
- User permissions
- Frequently requested read-only data

Do not cache data blindly.

Define:

Cache key
TTL
Invalidation strategy
Maximum cache size

---

26. URL & Routing Security

Sensitive information must not be exposed through URLs.

Avoid placing:

- Passwords

- Tokens
- API keys
- Sensitive personal information
- Internal identifiers where unnecessary

directly in query parameters.

For sensitive redirects or state:

- Use server-side validation.
- Use short-lived signed tokens where appropriate.
- Validate redirect destinations.
- Prevent open redirect vulnerabilities.
- Never trust client-controlled redirect URLs.

URL encoding is **not encryption**.

If actual confidentiality is required, use an appropriate cryptographic mechanism rather than simply Base64/URL encoding.

---

27. Authentication & Authorization

Authentication and authorization must be handled separately.

Authentication

Determines:

Who is the user?

Authorization

Determines:

What can the user access?

Implement:

- Secure login
- Session/token management

- Role-based access control
- Permission-based access control
- Route protection
- API-level authorization

Never depend only on frontend route protection.

Backend APIs must independently verify permissions.

---

28. Secrets & Environment Variables

Never hardcode:

API keys
Passwords
JWT secrets
Database credentials
Third-party credentials
Private tokens

Use environment variables or a secure secret manager.

Maintain separate configurations for:

Development
Testing
Staging
Production

---

29. Git & Repository Standards

The repository must have a proper `.gitignore`.

Never commit:

.env
.env.local

node_modules/
build/
dist/
coverage/
logs/
temporary files
IDE configuration
OS-generated files
credentials
private keys

Do not commit secrets even temporarily.

If a secret is accidentally committed, rotating the secret is required; deleting the file from the latest commit alone is not sufficient.

---

30. Git Branching Strategy

Use a consistent branching strategy.

Example:

main
develop
feature/*
bugfix/*
hotfix/*

Use meaningful branch names:

feature/client-search
feature/project-filter
bugfix/login-validation

Avoid:

test
new
final

final2
changes

---

31. Commit Standards

Commits should be small and meaningful.

Example:

feat: add client search
fix: resolve pagination issue
refactor: extract reusable filter component
perf: optimize project API
docs: update development guidelines

Avoid large commits containing unrelated changes.

---

32. Code Quality

Code should be:

- Readable
- Consistent
- Maintainable
- Modular
- Testable

Avoid:

- Extremely long functions
- Deep nesting
- Duplicate logic
- Unused imports
- Dead code
- Console logs in production
- Magic numbers
- Hardcoded configuration

Use linting and formatting tools consistently.

---

33. Performance Optimization

Performance should be considered during development rather than after completion.

Optimize:

- Bundle size
- JavaScript execution
- API requests
- Database queries
- Images
- Fonts
- Rendering
- Component re-renders
- Network requests

Use:

- Lazy loading
- Code splitting
- Memoization where justified
- Pagination
- Caching
- Image optimization
- Tree shaking
- Compression

Do not optimize blindly.

Measure first, then optimize the actual bottleneck.

---

34. Frontend Rendering Optimization

Avoid unnecessary re-renders.

Review:

- State placement
- Props changes
- Context usage
- Large lists
- Expensive calculations
- Effect dependencies

For large lists, consider virtualization where appropriate.

---

35. Loading States

Every asynchronous UI operation should have a proper loading state.

Examples:

- Page loading
- Table loading
- Dropdown loading
- Search loading
- File upload progress
- Button loading
- Modal loading

Avoid freezing the entire application when only one component is loading.

Prefer localized loading indicators.

---

36. Empty States

Every data-driven component must handle an empty response.

Examples:

No clients found.
No projects available.
No search results.
No records match the selected filters.

Empty states should be different from error states.

---

37. Error Boundaries & Global Error Handling

Frontend applications should have global error handling.

Handle:

- Unexpected rendering errors
- API errors
- Network failures
- Authentication expiration
- Permission errors

The application should fail gracefully instead of showing a blank screen.

---

38. Logging & Monitoring

Production logging must be structured.

Log important events such as:

- API failures
- Authentication failures
- Critical exceptions
- Background job failures
- Third-party service failures
- Performance issues

Never log:

- Passwords
- Tokens
- API keys
- Sensitive user information

---

39. Security Best Practices

Follow standard web security practices.

Protect against:

- XSS
- CSRF where applicable
- SQL injection
- Command injection
- SSRF
- Open redirects
- Broken access control
- Insecure file uploads
- Rate-limit abuse
- Credential exposure

Always validate and sanitize untrusted input where appropriate.

---

40. Rate Limiting

API endpoints must have appropriate rate limits.

Especially:

- Login
- Password reset
- Search
- File upload
- Public APIs
- Expensive AI/third-party APIs

Rate limits should be stricter for expensive or security-sensitive endpoints.

---

41. Third-Party API Management

Third-party APIs must be isolated behind backend services where appropriate.

Do not expose private API keys to the frontend.

Implement:

- Timeout
- Retry policy
- Error handling
- Rate-limit handling
- Fallback behavior
- Logging
- Response validation

Avoid calling third-party APIs unnecessarily.

---

42. Accessibility

The application should follow accessibility best practices.

Ensure:

- Proper semantic HTML
- Keyboard navigation
- Visible focus states
- Labels for inputs
- Accessible buttons
- Meaningful error messages
- Appropriate contrast
- Screen-reader-friendly structures

Do not rely only on color to communicate status.

---

43. Testing

Testing should be introduced during development.

Recommended levels:

Unit Tests
Integration Tests

API Tests
Component Tests
End-to-End Tests

Important functionality to test:

- Authentication
- Authorization
- Forms
- Validation
- Search
- Filters
- Pagination
- API errors
- File uploads
- Permissions
- Critical business logic

---

44. API Contract & Schema Validation

Frontend and backend must follow a defined API contract.

Document:

Endpoint
HTTP method
Request parameters
Request body
Response structure
Status codes
Error structure
Authentication requirements
Pagination
Filtering
Sorting

Use schema validation where appropriate.

Avoid silently changing API response structures.

Breaking API changes must be versioned or coordinated properly.

---

45. Date, Time & Timezone Handling

Use a consistent strategy for:

- Date storage
- Date formatting
- Timezone conversion
- API date formats

Prefer storing timestamps in a standard timezone such as UTC where appropriate, and convert them for display.

Never assume the user's timezone.

---

46. Accessibility & User Feedback

Every important user action should provide appropriate feedback.

Examples:

Save → Success message
Delete → Confirmation + result
Upload → Progress
API failure → Error message
Copy → Confirmation
Submit → Loading state

Avoid actions that appear to do nothing.

---

47. Confirmation for Destructive Actions

Actions such as:

Delete
Remove
Cancel
Deactivate
Reject
Permanent changes

should have appropriate confirmation when the action is destructive or difficult to reverse.

The confirmation message should clearly explain what will happen.

---

48. Production Build Standards

Before production deployment:

- Remove development logs.
- Remove unused dependencies.
- Verify environment variables.
- Run linting.
- Run tests.
- Build production bundle.
- Check bundle size.
- Verify API endpoints.
- Verify database configuration.
- Verify CORS.
- Verify authentication.
- Verify file uploads.
- Verify error handling.
- Verify monitoring.
- Verify security configuration.

---

49. Deployment Strategy

Use separate environments:

Development
↓

Testing
↓
Staging
↓
Production

Production should never be used as a development/testing environment.

Database migrations must be controlled and documented.

---

50. Production Checklist

Before releasing any feature:

Architecture

- Correct folder structure
- Clear component responsibility
- No unnecessary duplication
- Reusable components used

UI

- Consistent design
- Responsive
- Accessible
- Loading state implemented
- Empty state implemented
- Error state implemented

Forms

- Required fields validated
- Minimum limits defined
- Maximum limits defined
- Frontend validation
- Backend validation

API

- Pagination implemented
- Correct page size
- Server-side filtering
- Server-side sorting where required
- Request cancellation where appropriate
- Duplicate requests prevented
- Timeout configured
- Error handling implemented

Performance

- API response optimized
- Only required fields returned
- Database query optimized
- Egress minimized
- Images optimized
- Bundle checked
- Unnecessary API calls removed

Security

- No secrets in source code
- Authorization checked on backend
- File upload validated
- Input validated
- Sensitive URL parameters avoided
- Rate limiting considered
- Logs do not contain secrets

Git

- `.gitignore` verified
- No credentials committed
- Meaningful commit
- Branch follows naming convention
- Pull request reviewed

Testing

- Unit tests
- API tests
- Critical user flow tested
- Error scenarios tested
- Permission scenarios tested

51. Global Development Principles

The following principles apply to the entire application:

1. Reuse Before Creating

Before creating a new component, function, hook or utility, check whether an existing implementation can be reused.

2. Server Before Client

Perform filtering, sorting, pagination and data-intensive operations on the server whenever appropriate.

3. Minimum Data Transfer

Only transfer the data that is actually required.

4. One Standard

Similar functionality must follow one global implementation unless there is a documented reason to differ.

5. Secure by Default

Never assume the frontend is trusted.

6. Validate Everything

Validate user input and external data at appropriate boundaries.

7. Fail Gracefully

Every API and UI operation must have loading, success, empty and error handling where applicable.

8. Performance by Design

Performance must be considered during architecture and development, not only after production problems occur.

9. Avoid Premature Optimization

Measure actual bottlenecks before introducing complex optimization.

10. Keep It Simple

Prefer a simple, maintainable implementation over unnecessary complexity.

11. Single Source of Truth

Global configuration, constants, permissions, validation rules and reusable UI behavior should have one authoritative implementation.

12. Document Exceptions

If a module intentionally differs from the global standard, document the reason.

---

52. Definition of Done

A feature should not be considered complete merely because it works.

A feature is considered **production-ready** only when:

Functionality
+
UI Consistency
+
Validation
+
Security
+
Performance
+
API Optimization
+
Error Handling
+
Accessibility
+
Testing
+

Documentation
+
Code Quality
=
Production Ready

Every developer should follow these standards before marking a feature as completed.

Final Rule

Do not build each page independently. Build the application as one consistent system.

Every new page, component, API, hook, service and feature must reuse the application's existing standards wherever possible.

The goal is to create a system that is:

Scalable + Reusable + Secure + Performant + Maintainable + Consistent + Production Ready